UI-MOPD: Multi-Platform On-Policy Distillation for Continual GUI Agent Learning

Niu Lian, Alan Chen, Zhehao Yu, Chengzhen Duan, Fazhan Liu, Hui Liu, Pei Fu, Jian Luan, Yaowei Wang, Shu-Tao Xia, Jinpeng Wang

ABSTRACT

Recent advances in multimodal foundation models and agent systems have driven GUI agents from single-platform task execution toward cross-platform interaction. However, building multi-platform GUI agents remains challenging. On one hand, high-quality and executable cross-platform interaction trajectories are still scarce, and existing data often suffer from limited platform coverage. On the other hand, different platforms exhibit distinct interaction conventions, making joint or continual training prone to behavioral pattern mixing, platform-specific capability degradation, and catastrophic forgetting. To address these challenges, we construct Uni-GUI, a high-quality cross-platform GUI interaction dataset, and propose UI-MOPD, the first method that incorporates multi-teacher on-policy distillation into continual learning for GUI agents. UI-MOPD dynamically selects a platform-specific teacher according to the current environment and transfers platform-specific behavioral priors to a shared policy through platform-conditioned distillation, enabling adaptation to new platforms while preserving capabilities on existing ones. Experiments on OSWorld and MobileWorld show that UI-MOPD achieves task success rates of 38.2% and 12.0%, respectively, demonstrating its effectiveness in balancing cross-platform capability retention and new-platform adaptation. Project page: https://elispectre.github.io/UI-MOPD/.

About this document

notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel *and* used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at `https://github.com/ContextLab/llmXive`.

Automated review. See the accompanying `peer-review-llmxive.pdf` for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: `PROJ-1009-llmxive-follow-up-extending-ui-mopd-mult`.

UI-MOPD: Multi-Platform On-Policy Distillation for Continual GUI Agent Learning

Niu Lian[*1,3], **Alan Chen**[*4], **Zhehao Yu**[3], **Chengzhen Duan**[2], **Fazhan Liu**[2], **Hui Liu**[2], **Pei Fu**[2], **Jian Luan**[2], **Yaowei Wang**[3,5], **Shu-Tao Xia**[1,5], **Jinpeng Wang**[★3]

[1]Tsinghua Shenzhen International Graduate School, Tsinghua University, [2]Xiaomi

[3]Harbin Institute of Technology, Shenzhen, [4]Zhejiang University, [5]Peng Cheng Laboratory

[*]Equal contribution. [★]Corresponding author.

220110904@stu.hit.edu.cn (Niu Lian), wangjp26@gmail.com (Jinpeng Wang)

GitHub Hugging Face Homepage

Abstract

Recent advances in multimodal foundation models and agent systems have driven GUI agents from single-platform task execution toward cross-platform interaction. However, building multi-platform GUI agents remains challenging. On one hand, high-quality and executable cross-platform interaction trajectories are still scarce, and existing data often suffer from limited platform coverage. On the other hand, different platforms exhibit distinct interaction conventions, making joint or continual training prone to behavioral pattern mixing, platform-specific capability degradation, and catastrophic forgetting. To address these challenges, we construct **Uni-GUI**, a high-quality cross-platform GUI interaction dataset, and propose **UI-MOPD**, the first method that incorporates multi-teacher on-policy distillation into continual learning for GUI agents. UI-MOPD dynamically selects a platform-specific teacher according to the current environment and transfers platform-specific behavioral priors to a shared policy through platform-conditioned distillation, enabling adaptation to new platforms while preserving capabilities on existing ones. Experiments on OSWorld and MobileWorld show that UI-MOPD achieves task success rates of 38.2% and 12.0%, respectively, demonstrating its effectiveness in balancing cross-platform capability retention and new-platform adaptation.

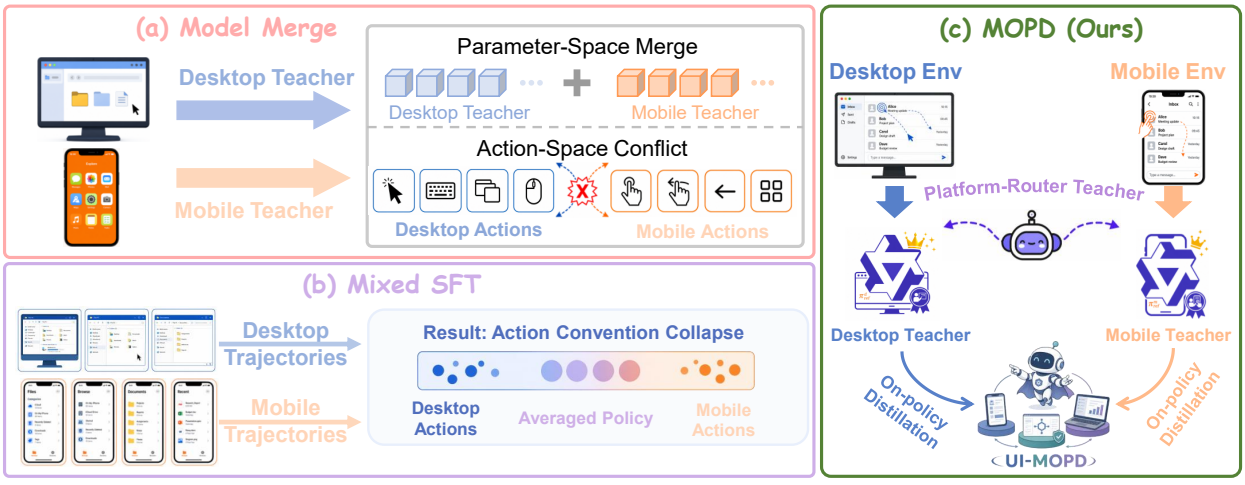

Figure 1 Motivation of UI-MOPD. Naively combining desktop and mobile signals, as in model merging or mixed SFT, can mix platform-specific behavioral conventions and produce an averaged policy. UI-MOPD uses platform-conditioned routing and multi-teacher on-policy distillation to integrate platform-specific expertise into a shared GUI agent.

Contents

1 Introduction

Recent advances in multimodal foundation models [1, 2, 4] have strengthened visual understanding, language reasoning, and tool-use capabilities [31]. Simultaneously, agent systems [8, 18, 34] have rapidly evolved from language only assistants toward interactive agents that can plan, invoke tools, and operate in external environments. Graphical user interface (GUI) agents [32, 41] have therefore emerged as a natural task form for connecting model intelligence with real digital environments. GUI agents must understand screen content, plan operation steps, and complete user goals through interface-level actions such as clicking, typing, and swiping. Early studies largely centered on platform-specific web navigation or computer automation. Benchmarks such as OSWorld [38] and MobileWorld [13] evaluate agents in interactive computer and mobile environments, driving a shift from static interface understanding toward long-horizon, platform-grounded interaction. Yet real-world workflows often span computer applications, mobile apps, and web services, requiring agents to adapt across heterogeneous GUI environments while preserving the interaction conventions of each platform. This progression raises a central question: *how can a shared GUI agent continually adapt across heterogeneous platforms while retaining platform-specific interaction behaviors*?

Recent efforts have attempted to extend GUI agents beyond isolated platforms mainly by scaling interaction data or training a shared policy on heterogeneous environments. Open-source datasets such as OpenCUA [33] and OpenMobile [7] provide growing collections of GUI trajectories, while multi-platform GUI agents [30, 41] typically combine signals from computer, mobile, or web environments through mixed supervised fine-tuning (SFT), mixed reinforcement learning (RL), or model merging. Although these efforts broaden platform coverage, they largely treat cross-platform learning as aggregating more data or merging heterogeneous training signals.

However, a capable cross-platform GUI agent requires more than exposure to multiple platforms. It must acquire transferable interface reasoning while preserving the distinct interaction conventions of each platform. This creates two bottlenecks. First, high-quality cross-platform trajectories remain scarce: existing datasets often focus on single-platform settings and may contain invalid actions, inaccurate state-action alignment, or inconsistent task granularity. Second, computer and mobile platforms differ in action semantics and affordances; for example, returning to the previous context may mean closing a window on computers but pressing the back button in a mobile app. Naively combining such signals can produce an overly averaged policy in joint training and, under continual learning, cause catastrophic forgetting of previously learned platform-specific behaviors. Thus, cross-platform GUI learning requires stable platform-specific behavioral anchors throughout policy optimization.

To address these challenges, we first build a unified cross-platform data collection harness that collects interaction data from computer and mobile environments with a consistent action interface and logging format. Using this harness, we collect approximately 110K and 50K interaction steps from computer and mobile environments, respectively. After rigorous filtering and quality control, we construct Uni-GUI, a dataset containing nearly 10K high-quality cross-platform interaction trajectories. Building on Uni-GUI, we propose UI-MOPD, the first method to introduce multi-teacher on-policy distillation (MOPD) into GUI agent continual learning, with platform-conditioned teachers for multi-platform adaptation.

UI-MOPD dynamically selects a platform-specific teacher for each rollout environment and transfers platform-specific behavior distributions to a shared policy. For computer and mobile environments, the model aligns with native interaction patterns preserved by corresponding teachers, preventing behavior signals with different interaction conventions from being indiscriminately mixed during optimization. These platform-specific teachers provide more than additional supervision. They serve as stable behavioral anchors, allowing a shared policy to improve task completion while preserving platform-specific behavioral priors. Through such environment-conditioned policy alignment, UI-MOPD better balances adaptation to new platforms with retention of existing platform behaviors, mitigates behavioral convention mixing and catastrophic forgetting of platform-specific behaviors, and encourages a platform-aware policy that activates different interaction modes according to environment context.

Empirical results further demonstrate the effectiveness of UI-MOPD. It achieves task success rates of **38.2**% on OSWorld and **12.0**% on MobileWorld, corresponding to relative improvements of **12.7**% and **55.8**%

over the base model, respectively. Importantly, these gains are observed in both computer and mobile environments, suggesting that UI-MOPD improves adaptation to new platforms without sacrificing existing platform capabilities and enables more balanced continual optimization across platforms.

The main contributions of this work are summarized as follows:

- We introduce **Uni-GUI**, a high-quality dataset for multi-platform GUI interaction. Using a unified cross-platform data-collection harness, we collect approximately 10K high-quality cross-platform interaction trajectories from computer and mobile environments.
- We propose **UI-MOPD**, the first method to introduce multi-teacher on-policy distillation for continual GUI agent learning, addressing behavioral convention mixing, platform-specific degradation, and catastrophic forgetting in continual GUI agent learning.
- We conduct systematic experiments on **OSWorld** and **MobileWorld**. **UI-MOPD** achieves task success rates of **38.2**% and **12.0**% on OSWorld and MobileWorld, respectively, demonstrating its effectiveness in preserving cross-platform capabilities while adapting to new platforms.

2 Related Work

2.1 GUI Agent: From Single-plantform to Multi-plantform

Graphical user interface (GUI) agents interpret natural language instructions and graphical user interfaces to automate humanz interaction tasks in digital environments. Early GUI agent [5] research has largely focused on single-platform settings. Web environments provide online evaluation for browser-based navigation [12, 53], where method such as WebVoyager improve web browsing. Mobile environments extend GUI agent evaluation to executable app control, cross-app workflows, personalization, and proactive assistance [6, 13, 23], with methods such as UI-R1 [19] and ClawGUI [28] exploring reinforcement learning. Desktop environments evaluate operating system level tasks [3, 38, 45], while recent computer-use agents such as EvoCUA [42] and ComputerRL [15] study learning and decomposition for desktop automation. Recent efforts on generalist GUI agents, such as MobileAgent-v3.5 [41], UI-Venus-1.5 [30], and UI-TARS-2 [32], have explored unified modeling and control across heterogeneous GUI platforms. In contrast, UI-MOPD focuses on continual learning for multi platform GUI agents, aiming to train an end to end model for long horizon navigation while preserving and transferring interaction capabilities across platforms.

2.2 Multi-Teacher On-Policy Distillation

During post training, different capabilities often exhibit a seesaw effect [27]: for example, mathematical RLVR may shorten reasoning traces and impair open ended writing, while each specialized training stage tends to improve one capability at the expense of others. To mitigate this issue, on policy distillation (OPD) [11, 46, 49] and on policy self distillation (OPSD) [20, 24, 51] have emerged as effective solutions. OPD samples trajectories from the student model and matches the teacher distribution along these trajectories through reverse KL divergence, thereby providing dense token level supervision. A natural extension of OPD is multi-teacher on-policy distillation (MOPD), which assigns the strongest checkpoint in each capability dimension as a teacher and enables the student model to absorb multiple capabilities in a single distillation stage. Recently, MOPD has been explored in foundation model post training. For example, MiMo-V2-Flash [37] and GLM-5 [50] use MOPD as the final post training step to distill a unified model from multiple expert models. Nemotron-Cascade 2 [47] uses MOPD as a forgetting recovery step between specialized RL stages, while DeepSeek-V4 [40] further employs full vocabulary logits, more than ten teacher models, and dedicated infrastructure for teacher scheduling and fault tolerant trajectory generation. However, MOPD remains largely unexplored in GUI agent. To our knowledge, UI-MOPD is the first to introduce MOPD into GUI agents. We further propose a platform conditioned distillation framework for multi platform GUI agents, enabling models to preserve and transfer interaction capabilities across different GUI platforms.

3 Method

3.1 Overview

UI-MOPD trains a unified native graphical user interface (GUI) agent that can adapt to two heterogeneous environments, desktop and mobile, while preserving platform-specific interaction behaviors. The construction of the unified cross-platform data collection harness and Uni-GUI is detailed in the Appendix. The training procedure contains two stages. In Stage 1, we perform **supervised fine-tuning (SFT)** of a vision-language foundation model on high-quality trajectories from each platform, yielding two expert teachers: a desktop teacher π^d_{ref} and a mobile teacher π^m_{ref}. In Stage 2, we employ **multi-teacher on-policy distillation (MOPD)** to integrate the two specialized capabilities into a unified student model π_θ.

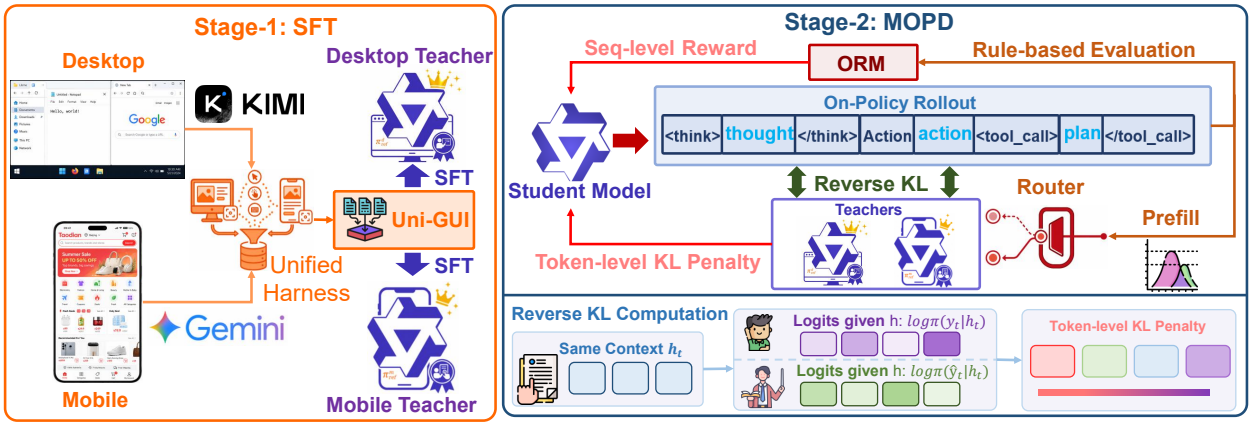

Figure 2 Overview of UI-MOPD training pipeline. In Stage 1, platform-specific desktop and mobile teachers are obtained by supervised fine-tuning on Uni-GUI trajectories collected from a unified cross-platform harness. In Stage 2, a shared student policy is trained with multi-teacher on-policy distillation, where platform-conditioned routing selects the corresponding teacher to provide reverse-KL guidance together with rule-based rollout rewards.

3.2 Multi-Teacher On-Policy Distillation

The core principle underlying UI-MOPD is to formulate multi-teacher knowledge integration as a conditional behavioral constraint during online policy optimization. Unlike directly merging multiple expert models or distilling from static offline trajectories, we let the student policy π_θ sample rollouts online from its current policy and introduce teacher supervision only on the states actually visited by the student. As a result, the distillation signal is concentrated on the state distribution where the current student policy truly makes decisions, rather than requiring the student to imitate all behavioral modes of the teacher policies. Specifically, teacher supervision is imposed in a platform-conditioned manner: rollouts from different platforms are aligned with their corresponding platform-specific teachers. In other words, teacher signals from different platforms are not simply aggregated into a single Kullback-Leibler (KL) penalty, but instead provide behavioral constraints according to the platform to which each rollout belongs.

This design changes the role of the KL term from a conservative regularizer, as commonly used in standard reinforcement learning from human feedback (RLHF) to limit policy drift, into a more targeted mechanism for transferring platform-specific expert interaction behaviors to a shared student policy during online reinforcement learning. Since teacher supervision is applied to the state distribution currently visited by the student, the resulting distillation signal is better matched to the student policy's current errors. This enables the shared policy to improve task success through reinforcement learning while preserving the platform-specific behavioral anchors of both desktop and mobile environments.

On-Policy Kullback–Leibler (KL). For the i-th sampled response $y^{(i)} = (y_1, \ldots, y_T)$, let $h_t^{(i)} = (x^{(i)}, y_{<t}^{(i)})$ denote the decoding state at token t. Given the platform routed teacher $\pi_{\text{ref}}^{(i)}$, we use the student-to-teacher on-policy KL

$$D_{\text{KL}}^{(t,i)} = D_{\text{KL}}\left(\pi_\theta(\cdot \mid h_t^{(i)}) \,\|\, \pi_{\text{ref}}^{(i)}(\cdot \mid h_t^{(i)})\right). \tag{1}$$

Equivalently,

$$D_{\mathrm{KL}}^{(t,i)} = \mathbb{E}_{a\sim\pi_\theta(\cdot|h_t^{(i)})}\left[\log \pi_\theta(a \mid h_t^{(i)}) - \log \pi_{\mathrm{ref}}^{(i)}(a \mid h_t^{(i)})\right]. \tag{2}$$

The mini-batch MOPD loss is

$$\mathcal{L}_{\mathrm{MOPD}}(\theta) = \frac{\sum_{i\in B}\sum_t m_t^{(i)}\mu^{(i)}\hat{D}_{\mathrm{KL}}^{(t,i)}}{\sum_{i\in B}\sum_t m_t^{(i)}\mu^{(i)}}, \tag{3}$$

where $m_t^{(i)}$ masks out prompt and padding tokens, $\mu^{(i)}$ is an adaptive KL mask, and $\hat{D}_{\mathrm{KL}}^{(t,i)}$ is a token level KL estimate.

K3 Estimator. Computing the full KL over the vocabulary is expensive. We therefore use the K3 estimator, which requires only the student and teacher log probabilities of the sampled token. Define

$$\delta_t^{(i)} = \log \pi_{\mathrm{ref}}^{(i)}(y_t \mid h_t^{(i)}) - \log \pi_\theta(y_t \mid h_t^{(i)}), \qquad \rho_t^{(i)} = \exp(\delta_t^{(i)}). \tag{4}$$

The token level estimate is

$$\hat{D}_{\mathrm{KL}}^{(t,i)} = \rho_t^{(i)} - \delta_t^{(i)} - 1. \tag{5}$$

This estimator is nonnegative, unbiased for $D_{\mathrm{KL}}(\pi_\theta\|\pi_{\mathrm{ref}})$ under samples from π_θ, and empirically lower variance than direct log-ratio estimators. We clamp $\delta_t^{(i)}$ in implementation for numerical stability.

Adaptive KL Masking. Teacher constraints are most useful when task feedback is still weak. For rollouts whose prompt group already receives sufficient reward, strong KL regularization may unnecessarily restrict exploration. We therefore use a group level mask

$$\mu^{(i)} = \begin{cases} 0, & \text{if } \frac{1}{G}\sum_{k\in g(i)} R^{(k)} > \tau_{\mathrm{KL}}, \\ 1, & \text{otherwise}, \end{cases} \tag{6}$$

where $g(i)$ denotes the prompt group of sample i. This mask removes the teacher penalty when task feedback is already sufficient for policy improvement, while preserving teacher guidance on low reward rollouts.

3.3 Platform-Conditioned Teacher Routing

Cross platform GUI interaction differs not only in visual layout, but also in action semantics, affordance structure, and execution context. Desktop tasks often require mouse operations, keyboard shortcuts, scrolling, and window switching, whereas mobile tasks rely on tap, swipe, long press, and application navigation. A single teacher or a direct mixture of teacher logits would compress heterogeneous behaviors into an averaged distribution, weakening platform specific interaction priors.

We instead train separate expert teachers and route each rollout by its platform label:

$$\pi_{\mathrm{ref}}^{(i)} = \begin{cases} \pi_{\mathrm{ref}}^{m}, & s_i \in \mathcal{S}_{\mathrm{mobile}}, \\ \pi_{\mathrm{ref}}^{d}, & s_i \in \mathcal{S}_{\mathrm{desktop}}, \end{cases} \tag{7}$$

where s_i is the data source label recorded during data construction. This routing affects only teacher log probability computation. The student remains a single shared policy, so UI-MOPD does not introduce multiple agents or additional teacher models at inference time.

During each reinforcement learning update, the student first samples rollouts from mixed platform prompts. The mini-batch is then partitioned by platform, each subset is evaluated by its corresponding teacher, and teacher log probabilities are merged back into the original batch order. The K3 estimator in Eq. 5 then provides token level distillation signals for Eq. 3. This procedure gives each platform a distinct behavioral anchor in the shared parameter space: reinforcement learning moves the policy toward higher task reward, while the routed teacher constrains this movement so that native interaction patterns are not overwritten by signals from another platform.

3.4 Reward Design

We use a structured outcome reward for GUI actions. The policy outputs an action JSON inside a tool-call format, including action type, coordinates, text, or other action specific fields. For each action type, we define a set of required dimensions, such as action type correctness, coordinate inclusion in a target bounding box, scroll direction, key set equality, or case insensitive text matching. Let $f_a \in [0, 1]$ be the fraction of matched dimensions for action a. The reward is

$$R(x, y) = \begin{cases} 1.0, & f_a = 1, \\ -0.5, & 0 \le f_a < 1, \\ -1.0, & \text{unparsable or invalid action.} \end{cases} \tag{8}$$

The intermediate penalty distinguishes partially valid but incorrect actions from invalid outputs, preserving a useful reward gap for group-relative advantage estimation used in policy optimization.

3.5 Training Objective

Stage-2 training combines clipped policy optimization with the platform conditioned MOPD penalty. Let

$$A_t^{(i)} = R(x^{(i)}, y^{(i)}) - \frac{1}{|g(i)|} \sum_{k \in g(i)} R(x^{(k)}, y^{(k)}) \tag{9}$$

be the token level advantage assigned to response tokens of sample i, where $g(i)$ denotes its prompt group. This advantage is computed from the structured reward in Eq. 8; we write it as A_t when the sample index is clear. Let

$$r_t(\theta) = \frac{\pi_\theta(y_t \mid h_t)}{\pi_{\theta_{\text{old}}}(y_t \mid h_t)}$$

be the policy ratio. For a rollout from platform p, we maximize the regularized objective

$$\mathcal{J}(\theta) = \mathbb{E}_{p,x,y\sim\pi_\theta} \left[\sum_t m_t \left(\ell_{\text{PG}}^{(t)}(\theta) - \beta\mu \, \hat{D}_{\text{KL}}^{(t,p)} \right) \right], \tag{10}$$

where

$$\ell_{\text{PG}}^{(t)}(\theta) = \min \left(r_t(\theta) A_t, \; \text{clip} \left(r_t(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}} \right) A_t \right). \tag{11}$$

Here $\hat{D}_{\text{KL}}^{(t,p)}$ is computed with the teacher selected by Eq. 7, m_t masks response tokens, μ is the adaptive KL mask, and β controls distillation strength. Equivalently, we minimize

$$\mathcal{L}(\theta) = -\mathcal{J}(\theta) = \mathcal{L}_{\text{PG}}(\theta) + \beta \mathcal{L}_{\text{MOPD}}(\theta). \tag{12}$$

This objective improves long horizon task completion through online reinforcement learning while preserving platform specific behavioral anchors through routed teacher supervision.

4 Experiments

4.1 Overview

We evaluate whether UI-MOPD can train a single native GUI agent that performs well across both desktop and mobile environments while preserving general GUI understanding ability. The experiments are organized around three questions: (i) whether UI-MOPD improves interactive task success on OSWorld [38] and MobileWorld [13], (ii) whether platform-conditioned distillation is more effective than direct mixed supervised fine-tuning or static model merging, and (iii) whether the resulting student maintains general GUI grounding, visual understanding, and static GUI agent evaluation capability.

Table 1 Baselines and integration strategies on OSWorld and MobileWorld. Missing evaluations are marked as "–".

Method	OSWorld	MobileWorld
General Models		
SeedVL-1.5 [10]	34.1%	–
Qwen3-VL-8B-Instruct [2]	33.9%	9.4%
Qwen3-VL-8B-Thinking [2]	33.9%	7.7%
Qwen3-VL-32B-Instruct [2]	32.6%	9.0%
Qwen3-VL-235B-A22B-Instruct [2]	31.6%	9.5%
Qwen3-VL-235B-A22B-Thinking [2]	38.1%	–
GUI Models (Single-Platform)		
OpenCUA-7B [33]	28.2%	–
OpenAI CUA o3 [21]	31.3%	–
OpenCUA-32B [33]	34.8%	–
GUI Models (Multi-Platform)		
UI-TARS-72B-DPO [22]	27.1%	–
UI-TARS-1.5-7B [22]	27.4%	–
GELab-Zero-4B [44]	31.9%	10.9%
GUI-Owl-7B [48]	34.9%	4.5%
GUI-Owl-32B [48]	–	5.5%
Integration Strategies		
Mixed-SFT	35.0%	6.4%
Model Merge (Weight Averaging) [35]	36.5%	6.8%
Model Merge (TIES Merging) [43]	36.8%	0%
UI-MOPD	**38.2%**	**12.0%**

4.2 Experimental Setup

Evaluation benchmarks. We evaluate the cross-platform navigation and task-execution capability of GUI agents on two interactive benchmarks. OSWorld [38] is used to evaluate desktop GUI task execution with 361 tasks, while MobileWorld [13] is used to evaluate mobile GUI task execution with 117 tasks. We additionally evaluate static GUI agent capability on AndroidControl* [17], where * denotes the evaluated subset and its construction details are provided in Appendix A. We assess GUI grounding ability on three grounding benchmarks: ScreenSpot-Pro [16], ScreenSpotV2 [36], and OSWorld-G [39]. We report task success rate as the main metric, together with the number of successful tasks over the total number of evaluated tasks.

Training procedure and models. The training procedure consists of two stages. In Stage 1, we perform supervised fine-tuning (SFT) of Qwen3-VL-32B-Thinking on Uni-GUI to obtain platform-specific expert teachers for desktop and mobile environments. In Stage 2, we train the student with reinforcement learning and multi-teacher on-policy distillation (MOPD). The student policy is initialized from Qwen3-VL-8B-Thinking and trained as a single shared policy with platform-conditioned teacher routing.

Implementation details. All training is implemented with verl [25], with Megatron-Core [26] as the training backend and SGLang [52] as the rollout engine. All experiments are conducted on 64 NVIDIA H100 GPUs, organized as 8 nodes with 8 GPUs per node. During data construction, desktop trajectories are collected with Kimi-K2.6 [29], while mobile trajectories are collected with Gemini-3.1-Pro [9]. We further use Gemini-3.1-Pro to clean the collected trajectories under a unified filtering pipeline. For evaluation, models are deployed and served with the vLLM [14] inference framework.

4.3 Main Results

Baselines. Table 1 compares UI-MOPD with representative baselines across four groups. **General Models** include general vision-language models evaluated directly on OSWorld and MobileWorld. **GUI Models (Single-Platform)** include GUI agents mainly specialized for a single platform, while **GUI Models (Multi-Platform)** include agents designed to operate across multiple GUI environments. We further include **Integration Strategies**

that build a single cross-platform policy from desktop and mobile supervision: Mixed-SFT jointly fine-tunes Qwen3-VL-8b-Thinking on mixed desktop and mobile data, while Model Merge combines platform-specific models through weight averaging or TIES merging.

Main analysis. Table 1 shows that UI-MOPD achieves the best balanced cross-platform performance among the compared methods, reaching 38.2% on OSWorld and 12.0% on MobileWorld. Compared with general models, UI-MOPD improves MobileWorld performance substantially over Qwen3-VL-8B-Thinking and Qwen3-VL-32B-Instruct, while also remaining competitive on OSWorld. Existing GUI-agent baselines are often strong on one side of the evaluation but incomplete or weaker on the other; for example, GELab-Zero-4B obtains 10.9% on MobileWorld but only 31.9% on OSWorld, whereas GUI-Owl-7B reaches 34.9% on OSWorld but drops to 4.5% on MobileWorld. The integration baselines further indicate that directly mixing heterogeneous supervision or statically merging platform-specific models is insufficient: Mixed-SFT improves neither platform consistently, and both model-merging variants underperform UI-MOPD, especially on MobileWorld. These results suggest that platform-conditioned on-policy distillation is more effective for transferring desktop and mobile expertise into a single shared GUI-agent policy.

4.4 Analysis of Cross-Platform Capability Transfer

Table 2 further examines whether a single student can retain capability across desktop and mobile environments. The platform-specific 32B teachers provide strong single-platform references, reaching 46.3% on OSWorld and 16.2% on MobileWorld. The goal of UI-MOPD is not to deploy multiple teacher models at inference time, but to transfer part of their platform-specific behavioral knowledge into a single 8B student.

Table 2 Teacher-student analysis on OSWorld and MobileWorld.

Method	OSWorld	MobileWorld
Qwen3-VL-8B-Thinking	33.9%	7.7%
Qwen3-VL-32B-Thinking	41.0%	9.4%
8B SFT on OSWorld	35.8%	0%
8B SFT on MobileWorld	35.8%	12.8%
Desktop Teacher, 32B	46.3%	–
Mobile Teacher, 32B	–	16.2%
UI-MOPD	**38.2%**	**12.0%**

The comparison highlights three findings. First, platform-specific SFT leads to unbalanced transfer. Fine-tuning the 8B model on OSWorld improves desktop performance from 33.9% to 35.8%, but its MobileWorld performance drops to 0%. Fine-tuning on MobileWorld improves mobile performance to 12.8%, but does not provide the same level of desktop improvement as UI-MOPD. This suggests that single-platform supervision can specialize the model toward one interaction style while weakening cross-platform robustness.

Second, UI-MOPD improves the 8B student on both platforms at the same time, reaching 38.2% on OSWorld and 12.0% on MobileWorld. Compared with the original 8B model, this corresponds to a gain of 4.3 points on OSWorld and 4.3 points on MobileWorld. It also outperforms the 32B base model on MobileWorld while using a smaller 8B student, indicating that the improvement does not simply come from model scale.

Third, UI-MOPD introduces platform-conditioned behavioral anchors during online policy optimization. Desktop rollouts are aligned with the desktop teacher, and mobile rollouts are aligned with the mobile teacher. This conditional distillation prevents heterogeneous interaction signals from being collapsed into a single averaged constraint, enabling the shared student to improve task success while maintaining platform-specific interaction patterns.

4.5 General GUI Static Understanding and Grounding Evaluation

We further evaluate whether cross-platform policy optimization preserves static GUI ability and GUI grounding capability. Table 3 reports results on AndroidControl*, ScreenSpot-Pro, ScreenSpotV2, and OSWorld-G.

Table 3 General GUI grounding, visual understanding, and AndroidControl* results. The star denotes the evaluated subset. The Model Merge row corresponds to the TIES-merging checkpoint.

Model	AndroidControl*	ScreenSpot-Pro	ScreenSpotV2	OSWorld-G
Qwen3-VL-8B-Thinking	78.73%	43.71%	91.27%	52.13%
Model Merge (TIES Merging)	74.01%	37.13%	88.60%	47.16%
UI-MOPD	**80.05%**	43.14%	90.88%	**52.84%**

Figure 3 Mobile task execution example of UI-MOPD.

On AndroidControl*, which is an evaluation subset sampled from AndroidControl, UI-MOPD achieves the best static mobile GUI performance among the three checkpoints. It improves overall accuracy from 78.73% for Qwen3-VL-8B-Thinking to 80.05%, while Model Merge drops to 74.01%. This indicates that MOPD better preserves mobile GUI understanding than static parameter merging when transferring cross-platform behavior into a shared student.

On the grounding benchmarks, UI-MOPD largely preserves the grounding ability of the base model. It obtains 43.14% on ScreenSpot-Pro and 90.88% on ScreenSpotV2, close to the base scores of 43.71% and 91.27%. It also slightly improves OSWorld-G from 52.13% to 52.84%. In contrast, Model Merge shows a clear decline across all three grounding datasets, dropping to 37.13% on ScreenSpot-Pro, 88.60% on ScreenSpotV2, and 47.16% on OSWorld-G. These results suggest that UI-MOPD is less destructive to GUI grounding than static parameter merging, while still improving interactive task performance on OSWorld and MobileWorld.

4.6 Case Study

Figure 3 presents a representative mobile GUI task executed by UI-MOPD. The example shows that the model can interpret the user instruction, identify the relevant UI elements on the screen, and produce a sequence of executable actions to complete the task. This qualitative result illustrates that UI-MOPD not only improves aggregate success rates, but also learns practical mobile interaction behaviors such as locating target widgets, navigating between screens, and grounding actions to appropriate screen regions. Additional desktop case studies are provided in Appendix E.

5 Conclusion

This work studies continual learning for cross-platform graphical user interface (GUI) agents in heterogeneous desktop and mobile environments. To address the behavioral pattern mixing, platform-specific capability degradation, and catastrophic forgetting that often arise in multi-platform training, we construct a unified cross-platform data-collection harness and use it to build Uni-GUI, a high-quality cross-platform graphical user interface interaction dataset. This provides a data foundation for learning executable trajectories across desktop and mobile environments. Methodologically, we propose UI-MOPD, which introduces multi-teacher on-policy distillation (MOPD) into cross-platform continual learning for GUI agents. The shared student policy samples rollouts from its current policy during reinforcement learning, and rollouts from different platforms are routed to their corresponding platform-specific teachers according to the source platform. Experimental results show that UI-MOPD achieves task success rates of 38.2% and 12.0% on OSWorld and MobileWorld, respectively, substantially outperforming conventional approaches such as model merging or distilling from static offline trajectories. These results demonstrate the ability of UI-MOPD to balance cross-platform capability retention with adaptation to new platforms. Overall, our findings suggest that introducing MOPD into multi-platform GUI agent training is effective in mitigating interference among heterogeneous interaction patterns and provides a feasible path toward unified GUI agents that can continually adapt to diverse digital environments.

References

[1] Anthropic. Introducing claude opus 4.6. Anthropic announcement, 2026. URL https://www.anthropic.com/news/claude-opus-4-6.

[2] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[3] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, et al. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.

[4] ByteDance Seed. Seed2.0 model card: Towards intelligence frontier for real-world complexity. *arXiv preprint arXiv:2603.11103*, 2026. URL https://arxiv.org/abs/2603.11103.

[5] Wanxia Cao, Chengzhen Duan, Pei Fu, Pengzhi Gao, Niu Lian, Fazhan Liu, Hui Liu, Heng Qu, Qinzhuo Wu, Zhehao Yu, Tongbo Chen, Shiqi Cui, Anan Du, Shukai Jia, Yuanfa Li, Wei Liu, Yike Liu, Wenchao Lu, Zhenbo Luo, Haoyuan Sun, Jiatong Sun, Cheng Tan, Yajie Wang, Changqiao Wu, Tao Xiong, Jiahui Yang, Yuxuan Yuan, Ruoceng Zhang, Shaojie Zhang, Jian Zhu, Jian Luan, and Cong Zou. Xiaomi-gui-0 technical report, 2026. URL https://arxiv.org/abs/2606.31410.

[6] Tongbo Chen, Zhengxi Lu, Zhan Xu, Guocheng Shao, Shaohan Zhao, Fei Tang, Yong Du, Kaitao Song, Yizhou Liu, Yuchen Yan, et al. Knowu-bench: Towards interactive, proactive, and personalized mobile agent evaluation. *arXiv preprint arXiv:2604.08455*, 2026.

[7] Kanzhi Cheng, Zehao Li, Zheng Ma, Nuo Chen, Jialin Cao, Qiushi Sun, Zichen Ding, Fangzhi Xu, Hang Yan, Jiajun Chen, et al. Openmobile: Building open mobile agents with task and trajectory synthesis. *arXiv preprint arXiv:2604.15093*, 2026.

[8] Yifan Duan, Qixiang Xu, Hengtao Wu, Zhanxun Liu, Wenhao Guan, Junxi Liu, Ziyang Ma, Kelu Xu, and Xie Chen. Audio-oscar: A multi-agent system for complex audio scene generation, orchestration, and refinement. *arXiv preprint arXiv:2606.07397*, 2026.

[9] Google DeepMind. Gemini 3.1 pro model card. Model card, 2026. URL https://deepmind.google/models/model-cards/gemini-3-1-pro/.

[10] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. Seed1.5-vl technical report. *arXiv preprint arXiv:2505.07062*, 2025.

[11] Wenjin Hou, Shangpin Peng, Weinong Wang, Zheng Ruan, Yue Zhang, Zhenglin Zhou, Mingqi Gao, Yifei Chen, Kaiqi Wang, Hongming Yang, et al. Uni-opd: Unifying on-policy distillation with a dual-perspective recipe. *arXiv preprint arXiv:2605.03677*, 2026.

[12] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, 2024.

[13] Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, et al. Mobileworld: Benchmarking autonomous mobile agents in agent-user interactive and mcp-augmented environments. *arXiv preprint arXiv:2512.19432*, 2025.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.

[15] Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. Computerrl: Scaling end-to-end online reinforcement learning for computer use agents. *arXiv preprint arXiv:2508.14040*, 2025.

[16] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 8778–8786, 2025.

[17] Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems*, 37: 92130–92154, 2024.

[18] Niu Lian, Yuting Wang, Hanshu Yao, Jinpeng Wang, Bin Chen, Yaowei Wang, Min Zhang, and Shu-Tao Xia. From verbatim to gist: Distilling pyramidal multimodal memory via semantic information bottleneck for long-horizon video agents. *arXiv preprint arXiv:2603.01455*, 2026.

[19] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Pengxiang Zhao, Guangyi Liu, et al. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 17608–17616, 2026.

[20] Zhengxi Lu, Zhiyuan Yao, Zhuowen Han, Zi-Han Wang, Jinyang Wu, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, et al. Self-distilled agentic reinforcement learning. *arXiv preprint arXiv:2605.15155*, 2026.

[21] OpenAI. Computer-using agent. https://openai.com/index/computer-using-agent/, January 2025. Accessed: 2026-07-02.

[22] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents, 2025. *URL https://arxiv.org/abs/2501.12326*, 2025.

[23] Chris Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations*, volume 2025, pages 406–441, 2025.

[24] Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

[25] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

[26] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[27] Mustafa Shukor, Louis Bethune, Dan Busbridge, David Grangier, Enrico Fini, Alaaeldin El-Nouby, and Pierre Ablin. Scaling laws for optimal data mixtures. *Advances in Neural Information Processing Systems*, 38:129554–129579, 2026.

[28] Fei Tang, Zhiqiong Lu, Boxuan Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Clawgui: A unified framework for training, evaluating, and deploying gui agents. *arXiv preprint arXiv:2604.11784*, 2026.

[29] Kimi Team, Yifan Bai, Yiping Bao, Y Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.

[30] Venus Team, Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, et al. Ui-venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*, 2026.

[31] Geng Tu, Dingming Li, Jun Huang, and Ruifeng Xu. Consensus-driven multi-agent cognitive reasoning for enhancing the emotional intelligence of large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 17751–17759, 2026.

[32] Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025.

[33] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Wu, et al. Opencua: Open foundations for computer-use agents. *Advances in Neural Information Processing Systems*, 38:139756–139806, 2026.

[34] Zixuan Wang, Yuchen Yan, Hongxing Li, Teng Pan, Dingming Li, Ruiqing Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Milestone-guided policy learning for long-horizon language agents. *arXiv preprint arXiv:2605.06078*, 2026.

[35] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pages 23965–23998. PMLR, 2022.

[36] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: Foundation action model for generalist gui agents. In *International Conference on Learning Representations*, volume 2025, pages 5090–5108, 2025.

[37] Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.

[38] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.

[39] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, et al. Scaling computer-use grounding via user interface decomposition and synthesis. *Advances in Neural Information Processing Systems*, 38, 2026.

[40] Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, et al. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.

[41] Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, et al. Mobile-agent-v3. 5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*, 2026.

[42] Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, et al. Evocua: Evolving computer use agents via learning from scalable synthetic experience. *arXiv preprint arXiv:2601.15876*, 2026. URL https://arxiv.org/abs/2601.15876.

[43] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. *Advances in neural information processing systems*, 36:7093–7115, 2023.

[44] Haolong Yan, Jia Wang, Xin Huang, Yeqing Shen, Ziyang Meng, Zhimin Fan, Kaijun Tan, Jin Gao, Lieyu Shi, Mi Yang, et al. Step-gui technical report. *arXiv preprint arXiv:2512.15431*, 2025.

[45] Pei Yang, Hai Ci, and Mike Zheng Shou. macosworld: A multilingual interactive benchmark for gui agents. *Advances in Neural Information Processing Systems*, 38:134014–134056, 2026.

[46] Wenkai Yang, Weijie Liu, Ruobing Xie, Kai Yang, Saiyong Yang, and Yankai Lin. Learning beyond teacher: Generalized on-policy distillation with reward extrapolation. *arXiv preprint arXiv:2602.12125*, 2026.

[47] Zhuolin Yang, Zihan Liu, Yang Chen, Wenliang Dai, Boxin Wang, Sheng-Chieh Lin, Chankyu Lee, Yangyi Chen, Dongfu Jiang, Jiafan He, et al. Nemotron-cascade 2: Post-training llms with cascade rl and multi-domain on-policy distillation. *arXiv preprint arXiv:2603.19220*, 2026.

[48] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. Mobile-agent-v3: Fundamental agents for gui automation, 2025. *URL https://arxiv.org/abs/2508.15144*, 4:21–27, 2025.

[49] Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026.

[50] Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

[51] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

[52] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

[53] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606, 2024.

Appendix

A Dataset Construction and Composition

AndroidControl⋆ Construction. To statically evaluate GUI understanding capability of UI-MOPD beyond interactive task execution, we construct an extracted AndroidControl subset, denoted as AndroidControl⋆. This subset contains 4,260 step-level records from 781 Android trajectories. Each record is stored in JSONL format and includes the trajectory identifier, step index, high-level task goal, per-step instruction, normalized action, screenshot path, and screenshot resolution. The referenced screenshots are provided as PNG files and are linked directly from the corresponding step records.

AndroidControl⋆ preserves the same normalized mobile action space used in Uni-GUI, including click, scroll, input_text, open_app, wait, navigate_back, long_press, and navigate_home. For actions that can be matched to a UI element, we also include grounding metadata such as target bounding boxes, widget class, visible text or content description, resource id, package name, ancestor information, and the number of matching UI nodes. Steps without a directly groundable target, such as waiting or app-level operations, leave the grounding field empty. This subset is mainly used to evaluate static mobile GUI understanding, verify action-screen alignment, and illustrate the format of mobile data after normalization.

Uni-GUI Composition. Uni-GUI is constructed from four groups of trajectories across desktop and mobile platforms, as summarized in Table 4. For each platform, we combine trajectories collected by our Unified Cross-Platform Data Collection Harness with cleaned open-source trajectories. On the desktop side, the self-collected portion contains about 95K interaction steps from desktop GUI environments, and the public portion contains about 13K cleaned steps from OpenCUA [33]. On the mobile side, the self-collected portion contains about 17K interaction steps from mobile GUI environments, and the public portion contains about 35K cleaned steps from OpenMobile [7]. In total, Uni-GUI contains approximately 160K steps and 11.5K trajectories.

We do not directly use raw public trajectories. Instead, OpenCUA and OpenMobile are processed with the trajectory cleaning and post-processing steps described in Appendix B, including action-space compatibility checking, trajectory filtering, and format normalization.

Table 4 Approximate composition of Uni-GUI. Counts are rounded for readability.

Platform	Source Type	Steps	Trajectories
Desktop	Self-collected	∼95K	∼7K
	OpenCUA	∼13K	∼0.8K
Mobile	Self-collected	∼17K	∼1K
	OpenMobile	∼35K	∼2.7K
Total		∼160K	∼11.5K

Including OpenCUA and OpenMobile broadens the coverage of GUI states, applications, and task types beyond the self-collected trajectories. These sources complement the desktop and mobile data collected by our harness, increase platform and application diversity, and provide additional supervision for cross-platform generalization after filtering and normalization.

B Unified Cross-Platform Data Collection Harness

Collecting high-quality GUI agent trajectories is challenging because the validity of a trajectory depends on the textual instruction, the current interface state, the executable action space, and the visual grounding of each interaction. A task that appears reasonable at the language level may still be unusable for training if the target UI element is not visible, the action cannot be represented by the student policy, or the instruction is inconsistent with the environment state. These issues become more pronounced in a cross-platform setting, where desktop and mobile environments use different observation formats, action primitives, and UI layouts.

To reduce such noise, the harness organizes data construction into four stages: query generation, trajectory collection, trajectory cleaning, and post-processing.

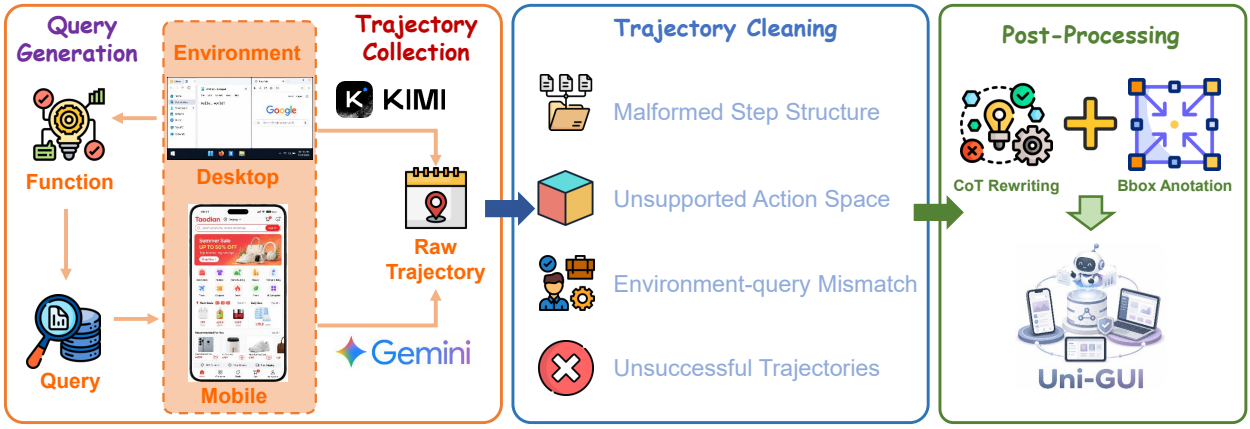

Figure 4 Overview of Unified Cross-Platform Data Collection Harness.

Query Generation. We first construct user queries from executable functionalities in the target environments. Instead of freely generating arbitrary instructions, we ask strong teacher models to identify realistic functional points supported by the corresponding desktop or mobile environment, and then synthesize natural user queries from these functional points. In our implementation, desktop queries are generated with the assistance of Kimi-K2.6, while mobile queries are generated with the assistance of Gemini-3.1-Pro. This environment-grounded query construction reduces the chance of collecting trajectories for underspecified, infeasible, or state-mismatched tasks.

Query Generation. We first construct environment-grounded user queries from real, executable functionalities in the target GUI environments. For the desktop domain, Kimi-K2.6 is used to assist functional-point extraction from the original initialized OSWorld [38] environments. For the mobile domain, Gemini-3.1-Pro is used to assist functional-point extraction from the original initialized MobileWorld [13] and AndroidWorld [23] environments. Instead of generating arbitrary instructions in free form, we ask the teacher models to identify realistic and usable functionalities supported by these environments. Based on the extracted functional points, we then synthesize user queries that are both natural and executable in the corresponding environments. This design reduces the likelihood of collecting trajectories for tasks that are underspecified, infeasible, or mismatched with the environment state.

Trajectory Collection. Given a generated query, the teacher model interacts with the target GUI environment to produce an execution trajectory. Each trajectory records the observations, actions, intermediate reasoning, and task states during the rollout. Desktop and mobile trajectories are collected through the same high-level interface, but the executed actions are kept in their native platform forms before later normalization. This allows the harness to retain platform-specific interaction patterns, such as desktop-oriented pointing and window operations or mobile-oriented touch and navigation behaviors, while keeping the collected data compatible with a unified training pipeline.

Trajectory Cleaning. Raw trajectories collected from GUI environments are noisy and cannot be directly used for Qwen-VL student-model training. We therefore apply a multi-stage cleaning pipeline. First, we remove trajectories with malformed step structures, such as non-contiguous step indices or duplicated steps. Second, we filter out trajectories whose actions cannot be mapped to the action space of the student model. This ensures that every retained demonstration can be faithfully imitated by the student. Third, we discard overly long trajectories with more than 40 steps, since such trajectories are more likely to contain inefficient exploration, accumulated errors, or ambiguous supervision signals. Fourth, we remove trajectories whose queries are inconsistent with the original environment or unsupported by the collected functional points.

Finally, we use Gemini-3.1-Pro as an automatic judge to check whether the trajectory successfully completes

the intended task, and only retain successful trajectories. Specifically, we decompose each task query into an ordered list of sub-tasks before judging. During trajectory inspection, the judge walks through the execution steps, identifies which sub-task each step corresponds to, and determines whether the corresponding sub-task has been completed. A trajectory is retained only when all sub-tasks are judged as completed. This sub-task-level adjudication avoids relying on a single long-context judgment over the entire trajectory, reduces the effect of noisy or redundant steps, and provides clearer attribution for failed executions.

Post-Processing. After cleaning, we convert the remaining trajectories into the training format required by the Qwen-VL-based policy. We first normalize the intermediate reasoning into a structured chain-of-thought format. This is necessary because the raw reasoning traces produced by Kimi-K2.6 and Gemini-3.1-Pro are often heterogeneous in structure, vary substantially in length, and differ from the reasoning style of Qwen3-VL models. Directly training on these unnormalized traces may disturb the student model's original output distribution. Therefore, we rewrite the reasoning traces into a consistent structure aligned with the expected Qwen3-VL input-output format.

We also re-annotate grounding bounding boxes for actions that refer to visual UI elements. These bounding boxes are used in the subsequent rule-based evaluation stage to determine whether the current action is executed on the correct target region. The final output of the harness is therefore a set of successful, executable, action-compatible, reasoning-normalized, and visually grounded GUI trajectories across both mobile and desktop platforms.

C Training and Implementation Details

C.1 Action Space and Trajectory Format

The prompt templates in Appendix F define the platform specific tool interfaces used by Qwen3-VL based policies. We summarize the corresponding action spaces in Table 5. Desktop trajectories use the computer_use interface with mouse and keyboard actions, while mobile trajectories use the mobile_use interface with touchscreen actions. During post processing, actions from different data sources are converted into these platform specific tool call formats so that every retained trajectory can be consumed by the training pipeline.

Table 5 Platform specific action spaces used in the trajectory format.

Platform	Tool	Actions
Desktop	computer_use	key, type, mouse_move, left_click, left_click_drag, right_click middle_click, double_click, triple_click, scroll, wait, terminate
Mobile	mobile_use	click, long_press, swipe, type, answer system_button, wait, ask_user, terminate

Each trajectory is stored as an episode directory. A mobile episode contains a normalized trajectory file task.json, a raw generation record task_raw.json, and screenshots indexed by step, such as 0.jpg and 1.jpg. The normalized task.json file stores episode metadata, including the task source, application name, application package, screen resolution, user query, episode identifier, device type, and train/test split. Its data field contains the step trajectory records. Each step records the step index, query, normalized reasoning, tool call action plan, screen resolution, grounding bounding boxes, screenshot path, and filtering or review flags.

The raw file task_raw.json is retained for traceability. It stores the original prompt, raw model response, raw action, converted action, and labeled screenshot reference for each step. This separation allows us to keep the original generation evidence while training only on the cleaned and normalized trajectory representation.

C.2 Training Hyperparameters

Table 6 summarizes the main training configuration used in our experiments. The student policy is initialized from Qwen3-VL-8B-Thinking, while the platform-specific teachers are initialized from Qwen3-VL-32B-Thinking.

Both teacher SFT and student training are run for one epoch. Student training uses a GRPO-based DAPO objective with multi-teacher on-policy distillation, where each prompt samples 8 rollouts and the OPD auxiliary KL loss is enabled with coefficient 0.01. For visual inputs, training uses only the current screenshot, while inference uses four historical screenshots, the current screenshot, and the full text action history. All training is conducted on 64 NVIDIA H100 GPUs organized as 8 nodes with 8 GPUs each, and asynchronous rollouts are served by SGLang.

Table 6 Training hyperparameters. TP, PP, and DP denote tensor, pipeline, and data parallelism, respectively.

Category	Hyperparameter	Value
Models and Training Epochs		
Student model	Initialization	`Qwen3-VL-8B-Thinking`
Teacher model	Initialization	`Qwen3-VL-32B-Thinking`
Teacher SFT	Epochs	1
Student training	Epochs	1
Infrastructure		
Cluster	Nodes / GPUs per node	8 / 8
Total GPUs	–	64 NVIDIA H100 GPUs
Parallelism		
Student 8B	TP / PP / DP	2 / 1 / 32
Teacher 32B	TP / DP	8 / 8
Rollout	TP	2
Batch Size and Sequence Length		
Training batch size	–	128
Generation batch size	–	384
Mini batch size	–	128
Micro batch size per GPU	–	4
Maximum prompt length	–	8192
Maximum response length	–	512
Visual Input		
Desktop image resolution	Desktop	1920×1080
Mobile image resolution	Mobile	1080×2400
Training visual context	Screenshots	Current screenshot only
Inference visual context	Screenshots	4 history screenshots + current screenshot
Inference text context	Action history	All previous text actions
Image pixel range	Min / max pixels	3,136 / 13,107,200
Optimization		
Learning rate	–	1×10^{-6}
Precision	–	bfloat16
Rollout samples per prompt	–	8
Clip ratio	Low / high / C	0.2 / 0.28 / 10.0
Loss aggregation	–	Token mean
OPD KL Loss		
KL loss type	–	k3
KL loss coefficient	–	0.01
Rollout Engine		
Engine	–	SGLang
Mode	–	Async
Temperature / top-p	–	1.0 / 1.0
GPU memory utilization	–	0.60
Maximum number of sequences	–	1024

D Fine-Grained Static GUI and Grounding Results

Table 7 Fine-grained results on AndroidControl⋆ and grounding benchmarks. AndroidControl⋆ denotes the evaluated subset. Base denotes Qwen3-VL-8B-Thinking, and Model Merge corresponds to the TIES-merging checkpoint.

Benchmark / Metric	Base	Model Merge (TIES)	UI-MOPD
AndroidControl⋆			
Action Type Accuracy	85.75%	81.62%	87.02%
Grounding (target)	88.04%	86.02%	88.33%
Grounding (ancestor)	89.59%	87.69%	89.98%
Overall Accuracy	78.73%	74.01%	80.05%
ScreenSpot-Pro			
Overall	43.71%	37.13%	43.14%
CAD	24.90%	20.31%	22.61%
Dev	41.81%	32.78%	41.14%
Creative	41.06%	35.48%	41.64%
Scientific	50.39%	50.79%	52.36%
Office	64.78%	49.57%	63.48%
OS	42.86%	36.73%	40.31%
ScreenSpotV2			
Overall	91.27%	88.60%	90.88%
mobile	93.41%	92.02%	91.62%
desktop	90.12%	88.02%	92.22%
web	89.70%	85.13%	89.02%
OSWorld-G			
Overall	52.13%	47.16%	52.84%
Text Matching	31.58%	47.37%	42.11%
Element Recognition	59.70%	47.76%	57.46%
Layout Understanding	56.44%	53.78%	60.44%
Fine-grained Manipulation	51.52%	48.48%	50.76%
Refusal	24.07%	14.81%	18.52%

Table 7 provides fine-grained results for the static GUI and grounding evaluations. On AndroidControl⋆, UI-MOPD improves all reported metrics over the base model, including action type prediction, target grounding, ancestor grounding, and overall accuracy. In contrast, Model Merge consistently degrades these metrics, suggesting that static parameter merging is less stable for preserving mobile GUI understanding.

For the grounding benchmarks, UI-MOPD largely preserves the base model's performance while avoiding the larger degradation observed in Model Merge. On ScreenSpot-Pro and ScreenSpotV2, UI-MOPD remains close to the base model and improves several subcategories such as Creative, Scientific, and desktop grounding. On OSWorld-G, UI-MOPD achieves the best overall score and improves layout understanding, indicating that multi-teacher on-policy distillation can retain fine-grained GUI grounding ability while improving cross-platform interactive performance.

E Additional Case Studies

Figure 5 shows an additional desktop GUI case study. The example illustrates that UI-MOPD can follow a multi-step desktop instruction, locate task-relevant regions in a dense interface, and execute mouse-and-keyboard actions according to the current screen state. Compared with mobile tasks, desktop tasks require handling larger layouts, window-level operations, and more precise cursor-based grounding. Together with the mobile case in Section 4.6, this example qualitatively supports the cross-platform capability of UI-MOPD.

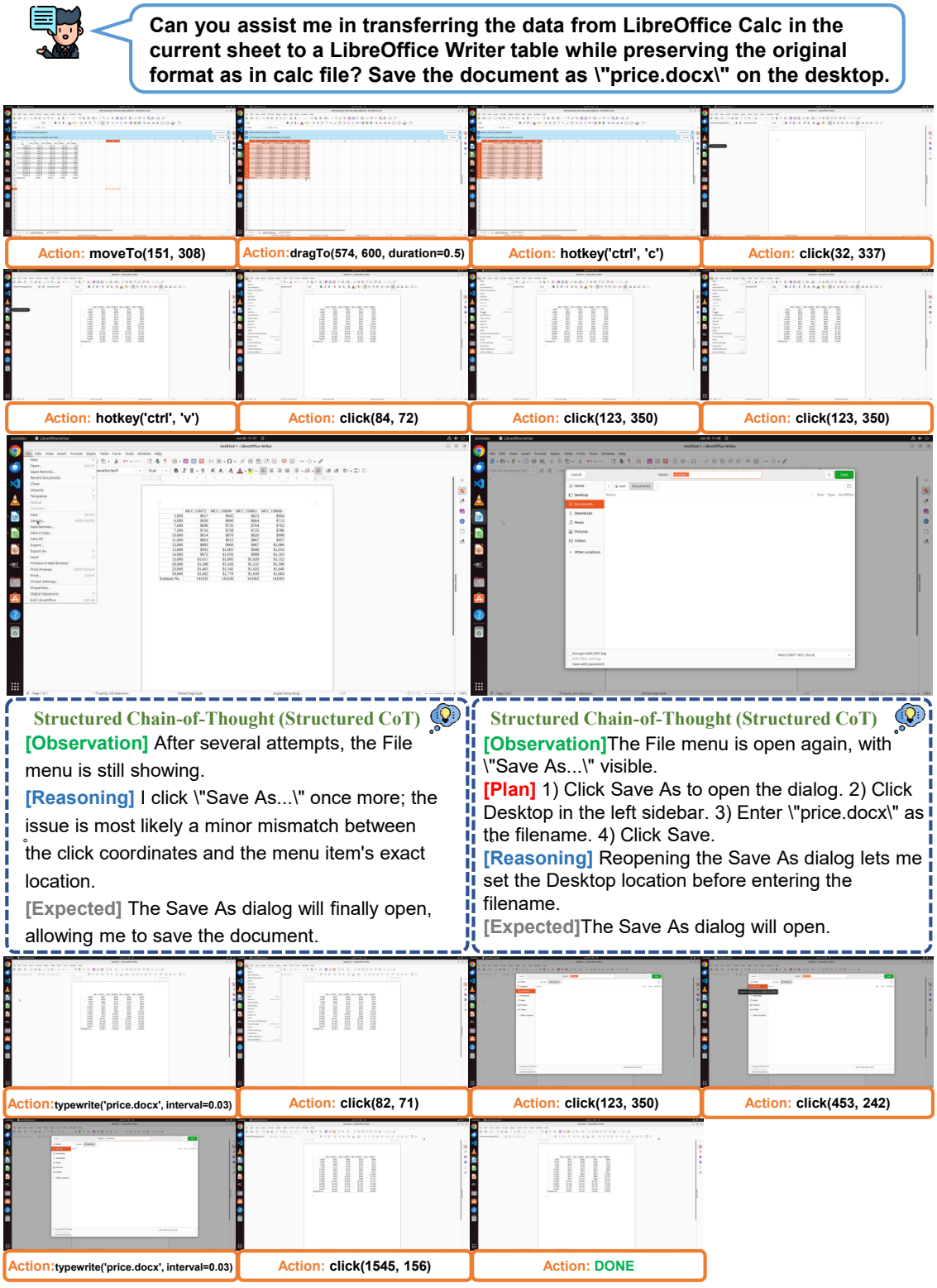

Figure 5 Desktop task execution example of UI-MOPD.

F Prompt Templates

We provide four system prompts used in our data construction and policy training pipeline. The first two prompts define the normalized desktop and mobile tool interfaces for Qwen3-VL-based policies, which are used during training and evaluation. The other two prompts are used for trajectory collection: Kimi-K2.6 is used to collect desktop trajectories, while Gemini-3.1-Pro is used to collect mobile trajectories before action normalization and post-processing.

Desktop System Prompt (Qwen3-VL)

```
# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{
  "type": "function",
  "function": {
    "name_for_human": "computer_use",
    "name": "computer_use",
    "description": "Use a mouse and keyboard to interact with a computer, and take screenshots. This is an
     interface to a desktop GUI. You do not have access to a terminal or applications menu. You must click
     on desktop icons to start applications. Some applications may take time to start or process actions, so
      you may need to wait and take successive screenshots. The screen resolution is 1000x1000. Consult
     screenshots before moving the cursor, and click UI elements with the cursor tip near the center.",
    "parameters": {
      "properties": {
        "action": {
          "enum": ["key", "type", "mouse_move", "left_click", "left_click_drag", "right_click", "
    middle_click", "double_click", "triple_click", "scroll", "wait", "terminate"]
        },
        "keys": {"description": "Required only by action=key."},
        "text": {"description": "Required only by action=type."},
        "coordinate": {"description": "The x,y coordinates for mouse actions."},
        "pixels": {"description": "The amount of scrolling."},
        "time": {"description": "The seconds to wait."},
        "status": {"enum": ["success", "failure"]}
      },
      "required": ["action"]
    }
  }
}
</tools>

For each function call, return a JSON object with function name and arguments within <tool_call></tool_call>
      XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>

# Response format

Response format for every step:
1) Action: a short imperative describing what to do in the UI.
2) A single <tool_call>...</tool_call> block containing only the JSON:
   {"name": <function-name>, "arguments": <args-json-object>}.

Rules:
- Output exactly in the order: Action, <tool_call>.
- Be brief: one sentence for Action.
- Do not output anything else outside those parts.
- If finishing, use action=terminate in the tool call.
```

Mobile System Prompt (Qwen3-VL)

```
# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{
  "type": "function",
  "function": {
    "name": "mobile_use",
    "description": "Use a touchscreen to interact with a mobile device, and take screenshots. This is an
     interface to a mobile device with touchscreen. You can perform actions like clicking, typing, swiping,
     etc. Some applications may take time to start or process actions, so you may need to wait and take
     successive screenshots. The screen resolution is 999x999. Click buttons, links, and icons near the
     center of the element.",
    "parameters": {
      "properties": {
        "action": {
          "enum": ["click", "long_press", "swipe", "type", "answer", "system_button", "wait", "ask_user", "
     terminate"]
        },
        "coordinate": {"description": "Required only by action=click, action=long_press, and action=swipe
     ."},
        "coordinate2": {"description": "Required only by action=swipe."},
        "text": {"description": "Required only by action=type, action=ask_user, and action=answer."},
        "time": {"description": "Required only by action=long_press and action=wait."},
        "button": {"enum": ["Back", "Home", "Menu", "Enter"]},
        "status": {"enum": ["success", "failure"]}
      },
      "required": ["action"]
    }
  }
}
</tools>

For each function call, return a JSON object with function name and arguments within <tool_call></tool_call>
      XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>

# Response format

Response format for every step:
1) Thought: one concise sentence explaining the next move.
2) Action: a short imperative describing what to do.
3) A single <tool_call>...</tool_call> block containing only the JSON:
   {"name": <function-name>, "arguments": <args-json-object>}.

Rules:
- Output exactly in the order: Thought, Action, <tool_call>.
- Be brief: one sentence for Thought, one sentence for Action.
- Do not output anything else outside those three parts.
- If finishing, use mobile_use with action=terminate in the tool call.
```

Desktop Trajectory Collection Prompt (Kimi-K2.6)

```
System Prompt

You are a GUI agent. You are given an instruction, a screenshot of the screen,
and your previous interactions with the computer. You need to perform a series
of actions to complete the task. The password of the computer is {password}.

For each step, provide your response in this format:
{thought}
## Action:
{action}
## Code:
{code}

In the code section, the code should be either pyautogui code or one of the
following functions wrapped in the code block:

- {
    "name": "computer.wait",
    "description": "Make the computer wait for 20 seconds for installation,
    running code, etc.",
    "parameters": {
      "type": "object",
      "properties": {},
      "required": []
    }
  }

- {
    "name": "computer.terminate",
    "description": "Terminate the current task and report its completion status",
    "parameters": {
      "type": "object",
      "properties": {
        "status": {
          "type": "string",
          "enum": ["success", "failure"],
          "description": "The status of the task"
        },
        "answer": {
          "type": "string",
          "description": "The answer of the task"
        }
      },
      "required": ["status"]
    }
  }
```

Mobile Trajectory Collection Prompt (Gemini-3.1-Pro)

```
# Role: Android Phone Operator AI

You are an AI that controls an Android phone to complete user requests.
Your responsibilities:
- Answer questions by retrieving information from the phone.
- Perform tasks by executing precise actions.

# Action Framework

Respond with exact JSON format for one of these actions:
| Action | Description | JSON Format Example |
| click | Tap visible element | {"action_type": "click", "coordinate": [x, y]} |
| double_tap | Double tap visible element | {"action_type": "double_tap", "coordinate": [x, y]} |
| long_press | Long press visible element | {"action_type": "long_press", "coordinate": [x, y]} |
| drag | Drag from one visible element to another | {"action_type": "drag", "start_coordinate": [x1, y1], "
    end_coordinate": [x2, y2]} |
| input_text | Type into field | {"action_type": "input_text", "text": "Hello"} |
| answer | Respond to user | {"action_type": "answer", "text": "It's 25 degrees today."} |
| navigate_home | Return to home screen | {"action_type": "navigate_home"} |
| navigate_back | Navigate back | {"action_type": "navigate_back"} |
| scroll | Scroll direction | {"action_type": "scroll", "direction": "down"} |
| status | Mark task as complete or infeasible | {"action_type": "status", "goal_status": "complete"} |
| wait | Wait for screen to update | {"action_type": "wait"} |
| ask_user | Ask user for information | {"action_type": "ask_user", "text": "what is the exact requirements
    do you need?"} |
| keyboard_enter | Press enter key | {"action_type": "keyboard_enter"} |
Note:
- The coordinate is the center of the element to be clicked, long pressed, or dragged.
- x and y are screen coordinates, with origin at the top left corner.
- x and y are normalized to [0, 1000].
# Execution Principles
1. Communication Rule:
- Always use the answer action to reply to users.
- Follow the user instruction strictly, e.g., only return a single number,
  only return True or False, or only return items separated by comma.
- Never use answer to indicate waiting or loading; use wait instead.
- The answer action terminates the task immediately.
2. Efficiency First:
- Choose the simplest path to complete tasks.
- If an action fails twice, try alternatives, e.g., long_press instead of click.
3. Smart Navigation:
- Gather information when needed.
- For scrolling, scroll direction is inverse to swipe direction.
- If scroll fails, try the opposite direction.
4. Text Operations:
- First click the input box to activate it before typing.
- For text manipulation, long press to select, use selection bar options,
  and delete by selecting then cutting.
5. Ask User:
- If there is not enough information to complete the task, use ask_user.
# Decision Process
1. Analyze goal, history, and current screen.
2. Determine if the task is already complete, and use status if true.
3. If not, choose the most appropriate action.
4. Output in the exact format below. The action must be a valid JSON string.
5. Only one tool call is allowed in one action.
# Expected Output Format

Thought: [Analysis including reference to key steps or points when applicable]
Action: [Single JSON action]

# Output Format Example
Thought: I need to type the weather question into the search box.
Action: {"action_type": "input_text", "text": "What is weather like in San Francisco today?"}
```